

Introducción

Definición de algoritmo:

- Conjunto de instrucciones finitas para resolver un problema
-

Recta de cocina:

- Input: Ingredientes
- Output: Comida
- Instrucciones: Algoritmo

Características:

- No son ambiguos: cada paso tiene un solo significado
- Entrada bien definida: Debe ser clara y consistente
- Salida bien definida: Debe indicar que tipo de output genera
- Finitos: Terminan en un tiempo determinado.
- Factibles: Pueden ser ejecutados con los recursos tecnológicos disponibles

¿Programa vs Algoritmo?

Un programa es la implementación de un algoritmo en un lenguaje de programación.

Representación de un algoritmo:

- Pseudo código
- Lenguaje natural
- Definición formal
- Diagramas de flujo

Complejidad:

- Cantidad de recursos utilizados para resolver una tarea (Tiempo/Memoria/Disco)
- Complejidad Temporal es la más común seguida por la complejidad espacial/memoria
- Meta: Reducir tiempo y memoria

Time complexity: Función del tiempo según el tamaño del input

$F(\text{input}) \rightarrow \text{tiempo}$

Tiempo se refiere a la cantidad de comparaciones, Accesos a mem, ... Operaciones elementales

- Space complexity: $f(\text{input}) \rightarrow \text{memoria usada}$
-

La(s) estructuras(s) de datos usadas en un algoritmo, tienen un efecto claro en la complejidad.

Análisis de algoritmos:

- Determinar tiempo y espacio requerido para ejecutar un algoritmo

¿Porqué analizar algoritmos?

- Predecir comportamiento
- Comparar distintos algoritmos para el mismo propósito.
- Optimizar
- Clasificar según complejidad

Tipos:

- Empírico
- Teórico

Análisis empírico

- Ejecutar un programa para evaluar el desempeño real
- Conocido como bench mark
- Fácil de entender y calcular
- No es independiente del hardware
- Requiere ejecutar el programa
- Muy utilizado en la práctica junto con pruebas A/B para identificar mejoras o regresiones.
- Se usa un enfoque de percentiles.

Percentil: Indica el % de valores que son inferiores a cierto valor D75, P95, P99

Usar promedios puede ser engañoso

Otra forma de análisis empírico es el 'profiling'. Es una técnica de análisis dinámico para encontrar cuellos de botella o secciones de la app con mal rendimiento. Provee una visión completa: CPU, rendering, caching.

Notas adicionales:

- El algoritmo es la abstracción del programa

Complejidad:

Se busca una function que crezca lento respecto al input, o que sea constante.

Las estructuras de datos evolucionan con el fin de mejorar el tema de la complejidad.

Big Data:

Surge cuando comienzan a haber fuentes de datos no estructurados.

Datos estructurados:

Tienen una estructura fija (como una tabla de Excel)

Datos no estructurados:

Se encuentran en redes sociales (Mensajes en json, comentarios en publicaciones...)

Es un conjunto de tecnologías que se usan con el fin de tratar cantidades descomunales de datos estructurados y no estructurados.

Empírico:

Algo que no tiene una formación básica. Ejemplo (Un ingeniero que aprendió todo por medio de tutoriales de youtube)

En el análisis empírico, se analiza el algoritmo ejecutándolo. Se utiliza mucho en la practica.

A/B testing:

Hay un grupo tratamiento y un grupo de control. Cuando se saca un feature, se realiza de manera controlada. A algunos usuarios se les muestra y a otros no. Se comparan los datos de la app/pagina en cada uno de los grupos y se va liberando a más grupos.

Percentiles:

Lo que se hace es buscar un umbral y se usa el porcentaje para definir para cuantos usuarios es eficiente y a cuantos no.

Análisis de algoritmos es determinar el tiempo y espacio requerido por un algoritmo para ejecutarse. El estudio de algoritmos se remonta al trabajo de *Charles Babbage* y de *Alan Turing*.

La máquina analítica de Babbage requería esfuerzo físico para configurarse, es por tanto entendible el interés de Babbage en la eficiencia de los algoritmos, dado que esto reduciría el esfuerzo físico requerido.

¿Por qué analizar algoritmos?

- Predecir el comportamiento
- Comparar distintos algoritmos para el mismo propósito (búsqueda, ordenamiento, etc.)
- Optimización
- Clasificar problemas según su complejidad

Análisis empírico de algoritmos

Ejecutar un programa paara evaluar el desempeño. Conocido también como *benchmark*.

- Es fácil de entender y calcular
- No es independiente de la máquina. No es lo mismo utilizar un procesador de hace 10 años que uno actual.
- Requiere ejecutar el programa.

Se utiliza mucho en la práctica profesional junto con pruebas A/B para evaluar regresiones o mejoras en una determinada pieza de software.

Se utiliza un enfoque de percentil para evaluar el rendimiento de un algoritmo.

Percentil se refiere a la posición de un valor en un conjunto de datos ordenados. Por ejemplo, el percentil 90 es el valor que es mayor que el 90% de los datos.

Otra forma de análisis empírico es el *profiling*. Es una técnica de análisis dinámico que permite encontrar “cuellos de botella” o secciones de la aplicación que consumen más recursos. Provee una visión muy completa: tiempo de ejecución, uso de memoria, uso de CPU, etc.

Análisis teórico de algoritmos

Parte de la teoría de complejidad computacional que provee estimación teórica de los recursos (tiempo, espacio) que un algoritmo requiere para ejecutarse.

- No depende de la máquina
- No es sencillo de calcular
- No requiere ejecutar el programa.
- Se enfoca en algoritmos y no en programas completos.

El output del análisis teórico es una función que describe el comportamiento del algoritmo. Por ejemplo, si se tiene un algoritmo de ordenamiento, se puede obtener una función que describe el tiempo que el algoritmo requiere para ordenar una lista de tamaño n .

- Si la función crece lento para valores grandes de n , el algoritmo es eficiente.

Enfoques de análisis teórico

- *Complejidad como función del input*: calcular una función contando las operaciones elementales que el algoritmo realiza.

Operaciones fundamentales son aquellas que se realizan en tiempo constante. Por ejemplo, una suma, una multiplicación, una comparación, etc.

- *Comportamiento asintótico*: sin calcular una función exacta, se busca una función que describa el comportamiento del algoritmo para valores grandes de n .

Complejidad como función del input

Se consideran operaciones fundamentales, es decir, operaciones que se realizan en tiempo constante. Por ejemplo, una suma, una multiplicación, una comparación, etc.

El tiempo que toma cada operación fundamental se designa con una constante T .

Por ejemplo, para el siguiente algoritmo:

```
int max(int[] array) {
    int max = array[0];
    for (int i = 1; i < array.length; i++) {
        if (array[i] > max) {
            max = array[i];
        }
    }
    return max;
}
```

La función que describe el tiempo que el algoritmo requiere para encontrar el máximo de un arreglo de tamaño n es:

```

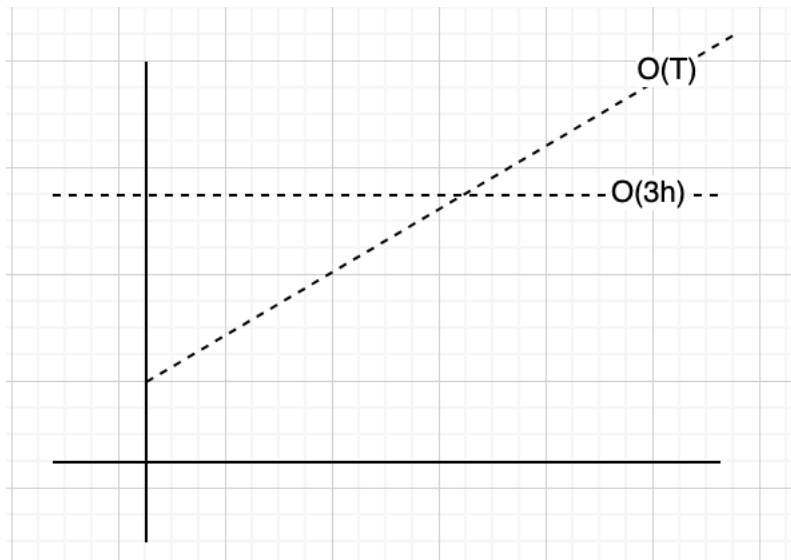
int max = array[0]; -----> 3T
for (int i = 1; i < array.length; i++) { ---> T + 2nT
    if (array[i] > max) { -----> 2T(n-1)
        max = array[i]; -----> 2T(n-1)
    }
}

```

Por lo tanto, $f(n) = 4T + 6nT$

Análisis asintótico

Suponga que usted necesita enviar un archivo a un amigo en Guanacaste. ¿Qué es más rápido, enviarlo por correo/FTP o llevarlo personalmente? Asumiendo que ir a Guanacaste sin presas, tarda siempre 3 horas, podríamos tener el siguiente gráfico:



No importa que tan grande sea el archivo, llevarlo físicamente siempre tarda lo mismo. Por medio electrónico, el tiempo de transferencia depende del tamaño del archivo y en algún momento será mayor que las 3 horas que tarda llevarlo físicamente.

Análisis asintótico busca encontrar la función que represente el crecimiento con respecto a n .

Eliminar las constantes

Considere la función que calculamos tiempo atrás: $f(n) = 4T + 6nT$. Análisis asintótico elimina los términos de menor relevancia, es decir, los que no son

significativos para el crecimiento de la función. De igual forma las constantes se eliminan. Por lo tanto, dicha función se puede expresar como

$$f(n) = O(n)$$

Lo que nos interesa en análisis asintótico, es la escalabilidad del algoritmo.

$$O(n^2 + n) \Rightarrow O(n^2)$$

$$O(n + \log(n)) \Rightarrow O(n)$$

$$O(5 * 2^n + 100n^2) \Rightarrow O(2^n)$$

Big O, Big Theta, Big Omega

Son notaciones para describir la ejecución de un algoritmo en términos de su comportamiento asintótico.

- *Big O*: describe el límite superior. Por ejemplo $O(n^2)$, $O(n)$, $O(2^n)$. El algoritmo es al menos tan rápido como este límite. No sobrepasa el límite dictado por *Big O*
- *Big Omega*: describe el límite inferior. Es decir, el algoritmo tendrá un comportamiento al menos tan lento como este límite.
- *Big Theta*: describe el comportamiento exacto del algoritmo. Es decir, el algoritmo se comporta exactamente como esta función. Es el límite ajustado, *Big O* y *Big Omega*.

De estas notaciones, la más usada es *Big O*

Mejor, peor y caso esperado/promedio

Formas de describir la ejecución del algoritmo. Por ejemplo, considerando la búsqueda en una lista enlazada sin ordenar:

- *Mejor caso*: $O(1)$ cuando el elemento buscado es el primero elemento consultado
- *Caso Promedio*: $O(n)$ dado que el elemento buscado puede estar en cualquier posición de la lista.
- *Caso peor*: $O(n)$ cuando el elemento buscado es el último elemento de la lista.

El mejor, peor y caso promedio, se puede describir con cualquiera de las notaciones de complejidad asintótica. Es incorrecto que *Big O* solo se refiere al peor caso, *Big Omega* es el mejor y *Big Theta* es el promedio.

Sobre la complejidad espacial

La cantidad de memoria que el algoritmo requiere. Se puede describir con Big-O, Big-Omega y Big-Theta también.

```
int sum(int n) {  
    if (n <= 0) {  
        return 0;  
    }  
    return n + sum(n-1);  
}
```

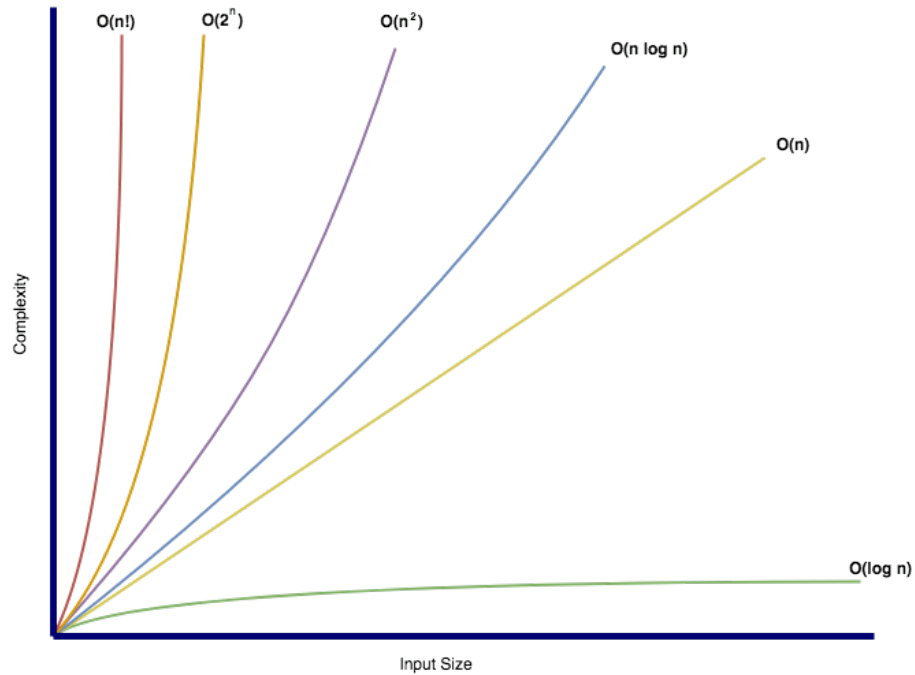
La complejidad espacial de este algoritmo es $O(n)$ dado que se requiere almacenar n llamadas recursivas en la pila de llamadas. La complejidad temporal es $O(n)$.

Ejercicio de complejidad espacial

```
int foo(int n) {  
    int sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum += bar(i);  
    }  
    return sum;  
}  
  
int bar(int a, int b) {  
    return a + b;  
}
```

No hay llamadas anidadas $\Rightarrow O(1)$

Big-O conocidas



Reglas generales para calcular complejidad espacial con Big-O

Sumar o multiplicar complejidades

Si el algoritmo es de la forma: haga x y luego y, entonces es una suma:

```
for (int a : arrayA) {  
    // Do something  
}  
for (int b : arrayB) {  
    // Do something  
}
```

La complejidad es $O(n) + O(m) = O(n+m)$ donde n es el tamaño de arrayA y m es el tamaño de arrayB.

De esto también podemos concluir que hacer dos iteraciones de un mismo array, sería: $O(n) + O(n) = O(2n) = O(n)$